

Care360: Engineering Report

An End-to-End IoT Platform for Elderly Health Monitoring

Executive Summary

Care360 is a four-tier IoT system designed to monitor the vital signs of elderly residents in community care settings. A wrist-worn smartwatch collects biometric data (heart rate, accelerometer), streams it via Bluetooth Low Energy (BLE) to a Raspberry Pi gateway, which performs local anomaly detection before forwarding only the resulting alerts to AWS IoT Core. A serverless Lambda pipeline scores, persists, and dispatches each alert to community-centre staff via email (SNS).

The system embodies a strict data-boundary principle: raw physiological readings never leave the local network. Only locally-detected anomaly events are uploaded to the cloud. This minimises privacy exposure, bandwidth, and cloud costs while keeping clinically meaningful events observable to caregivers.

Total codebase: ~1,100 LOC across Kotlin (Android Wear OS), Python (Raspberry Pi + AWS Lambda), and CloudFormation/SAM YAML.

1. Project Context and Goals

1.1 Problem Statement

Community care centres for the elderly face a recurring monitoring gap: residents living alone are at risk of undetected medical events (cardiac arrhythmia, falls, residential fires). Continuous one-to-one human supervision is economically infeasible, yet commodity wearables typically either (a) lock data into proprietary clouds with privacy concerns, or (b) lack the alerting workflows that caregivers require.

Care360 targets a middle ground: a self-hosted edge gateway that performs local detection, with the cloud used only as an alerting and audit-trail backbone.

1.2 Functional Requirements

- Continuous biometric collection from a smartwatch worn 24×7.
- Detection of two classes of critical events:

Cardiac anomalies — heart rate sustained outside a configurable range.
Fire detection — flame sensor triggered in the resident's home.

Real-time alerts to community-centre staff (email).

Audit history of all alerts for 90 days.

Resilience to transient BLE disconnection, network outages, and device reboots.

1.3 Non-Functional Requirements

Privacy: raw biometric data never leaves the local network.

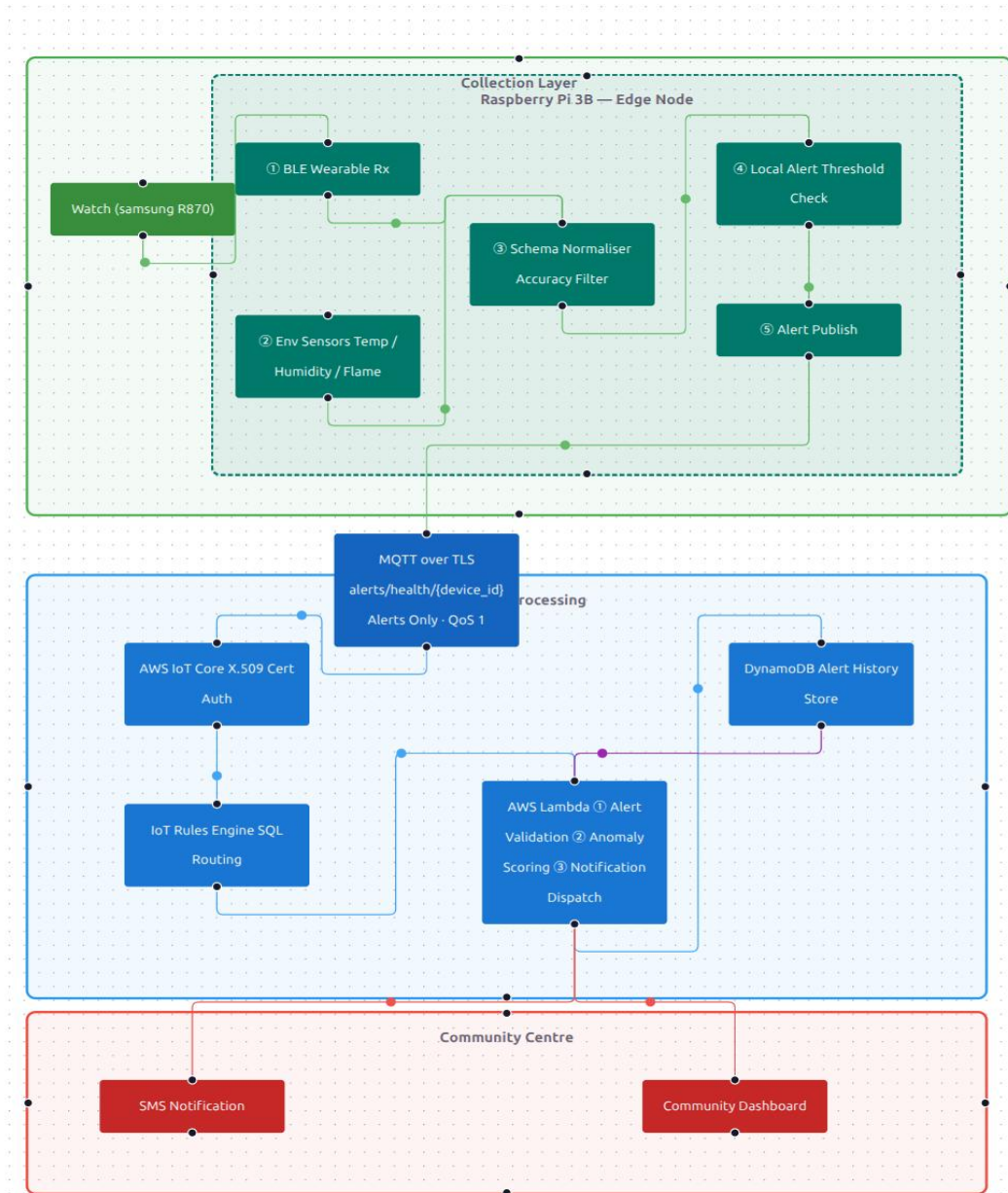
Availability: unattended operation; automatic restart after power loss.

Low operational cost: pay-per-use AWS services, free-tier-friendly footprint.

Reproducible deployment: infrastructure-as-code (SAM/CloudFormation).

2. System Architecture

2.1 Four-Tier Overview



2.2

Component Responsibilities

Tier	Component	Primary Responsibility
1	Wear OS App (Kotlin)	Sensor collection, batch packing, BLE transmission
2	Pi Gateway (Python, bleak, paho-mqtt)	BLE reception, environmental sensing,

		anomaly detection, MQTT publish
3	AWS Cloud (Lambda + DynamoDB + SNS)	Alert validation, severity scoring, persistence, dispatch
4	Dashboard (FastAPI + React)	Historical visualisation, configuration UI (work in progress)

2.3 Repository Layout

```
care360/
├── DATA_CONTRACT.md      # Wire-format contract between watch and Pi
├── wear_os/               # Tier 1: Android Wear OS app (Kotlin)
├── pi-gateway/            # Tier 2: Raspberry Pi gateway (Python)
├── cloud/                 # Tier 3: AWS SAM infrastructure + Lambda
├── dashboard/            # Tier 4: FastAPI backend + React frontend
└── flow-diagram/         # Architecture diagrams (Vite)
```

3. Data Contract

Care360 explicitly separates the wire format from the implementation. DATA_CONTRACT.md defines the JSON schema that the watch and Pi must both honour. Any change requires synchronised updates on both ends.

3.1 Telemetry Batch (Watch → Pi)

```
{
  "device_id": "gw4_hz_01",
  "sys_timestamp": 1716354162000,
  "battery_level": 82,
  "payload": [
    { "sensor": "HEART_RATE", "value": 76.0, "accuracy": 3, "offset_ms":
0    },
    { "sensor": "HEART_RATE", "value": 78.0, "accuracy": 3, "offset_ms":
2000 },
    { "sensor": "ACCELEROMETER", "values": [0.15, 9.81, -0.42], "accuracy": 3,
"offset_ms": 1500 }
  ]
}
```

3.2 Key Design Decisions

Decision	Rationale
sys_timestamp + offset_ms (instead of per-reading absolute timestamps)	Watch wall-clock is unreliable (no NTP). Batch-relative offsets are guaranteed monotonic and let the Pi reconstruct intra-batch ordering for sliding-window analysis.
accuracy field on every reading; readings with accuracy < 2	Aligns with SensorManager's accuracy levels. Filters out readings

discarded	from a loose-fitting watch (a common false-alarm source).
value vs values mutually exclusive	Heart rate is scalar; accelerometer is a 3-vector. Mutually exclusive fields keep the schema "flat" without a tagged union.
JSON over BLE (not binary)	Human-readable for debugging; trivial to log; 5-second batches are small enough that JSON overhead is negligible.
One BLE Notify = one complete JSON batch	Never split a JSON object across BLE packets; if a batch exceeds the 490-byte threshold (MTU 512 – ATT overhead), the application splits at the payload array boundary.

3.3 MQTT Topic Convention

alerts/health/{device_id} QoS 1 Pi → AWS IoT Core

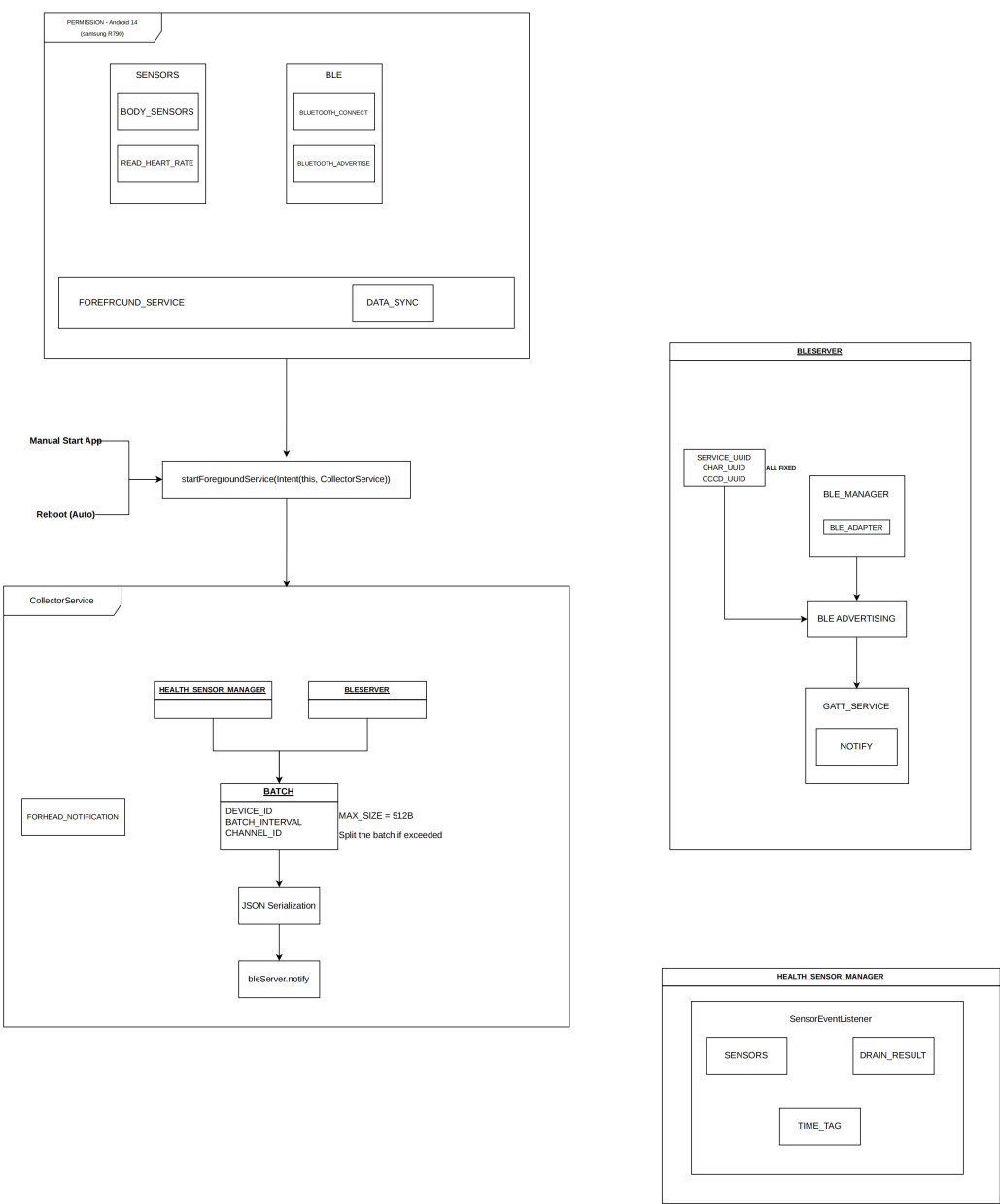
The wildcard alerts/health/+ is consumed by a single IoT Rule that forwards all device alerts to the Lambda handler.

4. Tier 1: Smartwatch (Wear OS, Kotlin)

4.1 Module Inventory

File	LOC	Role
AndroidManifest.xml	70	Permissions, foreground-service declaration, device features
presentation/MainActivity.kt	74	Runtime-permission "cashier"; launches the foreground service
BootReceiver.kt	13	Restarts service after device reboot
CollectorService.kt	160	Orchestrator: 5-second batch loop, JSON serialisation, BLE notify
sensor/HealthSensorManager.kt	86	SensorEventListener for heart rate + accelerometer; buffer + drain
data/SensorReading.kt	35	Single-reading data model with custom equals/hashCode for FloatArray
data/TelemetryBatch.kt	12	Batch wrapper: deviceId + sysTimestamp + payload
ble/BleGattServer.kt	159	GATT Server with Notify characteristic, BLE advertising

4.2 Component Architecture



4.3 Key Engineering Choices

Foreground Service with dataSync type. Android 14 forbids long-running background services. Declaring `foregroundServiceType="dataSync"` plus the matching permission (`FOREGROUND_SERVICE_DATA_SYNC`) plus a low-importance ongoing notification (`IMPORTANCE_LOW`) is the only legal way to keep the collector alive after screen-off.

Two entry points converging on one service. Both `MainActivity` (user tap) and `BootReceiver` (`BOOT_COMPLETED` broadcast) call `startForegroundService(...)` for the same `CollectorService`. The service's `onCreate` runs once; subsequent starts only re-invoke `onStartCommand`. This guarantees the collector resumes after a power-on without user intervention.

Coroutine-driven batch loop. A `SupervisorJob`-scoped coroutine on `Dispatchers.Default` ticks every 5 s, drains the sensor buffer, serialises to JSON, and notifies via BLE. `Dispatchers.Default` (CPU pool) is chosen over `Dispatchers.IO` because the work is JSON formatting + a non-blocking BLE syscall, not blocking I/O. The while (`isActive`) predicate ensures clean cancellation on `scope.cancel()`.

Software downsampling of accelerometer. Android's `SENSOR_DELAY_NORMAL` delivers ~5 Hz on accelerometers; heart rate is much slower. To prevent accelerometer readings from dominating each batch and to balance information density, an application-level filter rejects accelerometer events arriving within 500 ms of the previous one (effective 2 Hz).

Sensor object reuse pitfall. `SensorEvent` is reused by the Android framework: holding a reference to `event.values` causes silent data corruption on the next callback. The code defensively copies via `event.values.copyOf()` for the accelerometer array.

BLE role: watch as GATT Server, Pi as Client. Per BLE convention, the battery-powered, lower-compute device acts as Peripheral (advertises + waits). The mains-powered gateway acts as Central (scans + connects). Reversing the roles would force the watch to scan continuously, draining its battery.

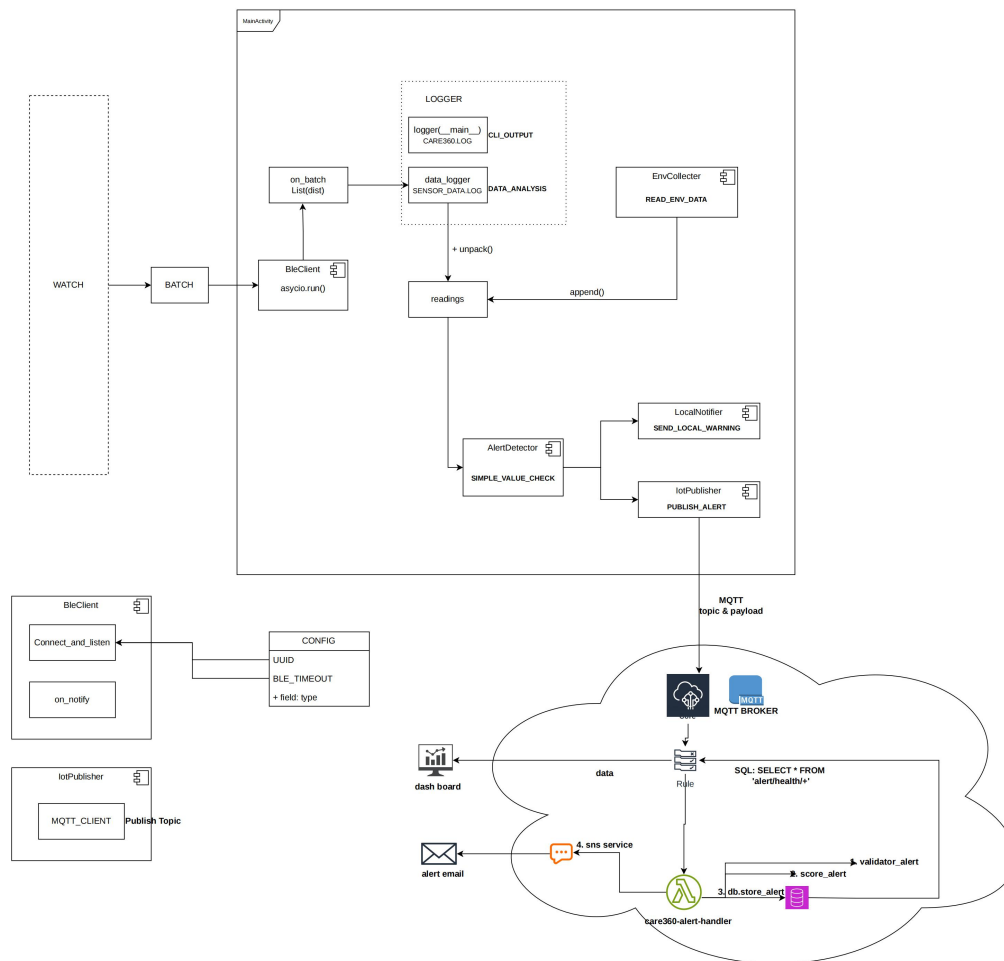
31-byte advertising packet constraint. A 128-bit custom Service UUID occupies 18 bytes of the 31-byte advertising payload. The device name is intentionally suppressed (`setIncludeDeviceName(false)`) so the UUID fits; the Pi discovers the watch by filtering on this UUID rather than by name. Recursive payload splitting (not byte splitting). When a serialised batch exceeds 490 bytes, the code recursively halves the payload array and re-serialises each half. Splitting at byte boundaries would yield malformed JSON on the Pi.

5. Tier 2: Raspberry Pi Gateway (Python)

5.1 Module Inventory

File	LOC	Role
main.py	87	Entry point; orchestrates components via on_batch callback closure
config.py	17	Constants: DEVICE_ID, BLE UUIDs, HR thresholds, AWS endpoints
ble/ble_client.py	65	bleak-based BLE Central with automatic reconnection
pipeline/schema_packager.py	43	Implements the data-contract parsing rules
pipeline/local_alert.py	60	Stateful anomaly detectors
sensors/env_collector.py	65	Sense HAT + GPIO mock for environmental sensors
alerts/notifier.py	80	Local audible + log notification
mqtt/iot_publisher.py	53	Paho-MQTT client to AWS IoT Core (mTLS, QoS 1)

5.2 Runtime Architecture



5.3 Key Engineering Choices

Inversion of control via a closure callback. `main.py` defines an `on_batch(batch)` function that closes over locally-instantiated components (`env`, `detector`, `notifier`, `publisher`) and passes it to `WatchBleClient`. The BLE layer is agnostic to business logic; the orchestrator is agnostic to BLE protocol details. Both can be tested independently with mock counterparts.

Two-logger separation: operational vs data. The root logger writes human-readable events to `stdout` and `care360.log` at INFO level. A dedicated "data" logger writes raw JSON batches (one per line) to `sensor_data.log` with `propagate = False` to prevent contamination of the operational stream. This makes `sensor_data.log` directly consumable by `jq` or `pandas` for offline analysis.

Three-layer error containment in the BLE client. `_on_notify` catches JSON/UTF-8 errors and discards malformed batches. `_connect_and_listen` raises on scan timeout. `run_forever` catches all exceptions, logs a warning, sleeps 5 s, and retries. The result: a transient signal disturbance does not break the BLE connection; a peripheral disappearance does not crash the gateway.

A subtle struct alignment hotfix. On 64-bit ARM, `sense-emu`'s use of the native `@ struct` format introduces padding bytes that shift the byte layout, corrupting humidity/pressure/IMU reads. The fix is a monkey-patch that rebuilds the structs with the `=` (no-padding) format. The HOTFIX comment ensures the patch can be removed when upstream fixes the bug.

mTLS with per-device X.509 certificates. The MQTT client authenticates to AWS IoT Core using a device-specific certificate + private key, validated against the Amazon Root CA. `client_id = DEVICE_ID` ensures one identity per gateway; `clean_session = False` allows the broker to queue alerts during transient disconnections.

6. Tier 3: AWS Cloud

6.1 Infrastructure (SAM / CloudFormation)

template.yaml provisions five AWS resources:

Resources:

AlertTable: # DynamoDB — alerts history (device_id+timestamp composite key, 90-day TTL)

AlertTopic: # SNS topic — fan-out destination

EmailSubscription: # SNS subscription — community-centre email

AlertFunction: # Lambda — handler.handler (Python 3.12, 128 MB, 30 s timeout)

IoTAlertRule: # IoT Rule — SELECT * FROM 'alerts/health/+' → invoke Lambda

Deployment is a single command: ./deploy.sh builds the Lambda artefact and runs sam deploy with parameter overrides for the notification email.

► SAM template

CloudFormation stack

Resources (7)

Last fetched 24/05/2026, 16:48:09

Q Search by attributes or search by keyword

< 1 >

⚙

Logical ID ▲	Physical ID	Type ▼	Last modified ▼
► AlertFunction	care360-alert-handler	Lambda Function	23 hours ago
AlertTable	care360-alerts	DynamoDB Table	1 day ago
AlertTopic	arn:aws:sns:ap-southeast-2:531855102820:care360-alerts	SNS Topic	1 day ago
EmailSubscription	arn:aws:sns:ap-southeast-2:531855102820:care360-alerts:782fdc8f-82e9-466c-9570-746208872a5b	SNS Subscription	1 day ago
IoTAlertRule	care360AlertRule	IoT TopicRule	1 day ago

6.2 Lambda Pipeline

IoT Rule fires



handler.handler(event, context) (cloud/lambda/alert_handler/handler.py)



- validate_alert(event) Type/format checks; reject malformed
- severity = score_alert(event) Returns 1 (LOW), 2 (MEDIUM), or 3 (CRITICAL)
- event["severity"] = severity
- event["processed_at"] = now_ms
- store_alert(event) put_item to DynamoDB; ttl = ts + 90 days
- dispatch(event, severity) SNS publish with templated email body

SQL statement

SQL statement
SELECT * FROM 'alerts/health/+'

SQL version
2016-03-23

Actions

Error action

Tags

Actions (1)

View details

Actions occur when an event is triggered. Actions are executed until all actions are completed or an error occurs. To add or remove actions, you will need to edit the rule.

Service	Action
☐ Lambda	Send a message to a Lambda function

care360-alert-handler

Throttle

Copy ARN

Actions

This function belongs to an application. [Click here](#) to manage it.

Function overview

info

Diagram

Template

care360-alert-handler

Layers (0)

AWS IoT

+ Add trigger

+ Add destination

Export to Infrastructure Composer

Download

Function ARN
arn:aws:lambda:ap-southeast-2:5318551028:20:function:care360-alert-handler

Description
-

Last modified
23 hours ago

Application
[care360-cloud](#)

6.3 Module Details

File	Role
handler.py	Entry point. Linear pipeline: validate → score → enrich → store → dispatch.
validator.py	Schema validation. Rejects missing fields, unknown alert types, malformed device_id (regex ^gw4_[a-z]+\d+\$), non-numeric timestamps.
scorer.py	Severity rule: FLAME_DETECTED → 3; heart rate scaled by deviation from 45-120 range (<30/>150 → 3, <40/>130 → 2, else 1).
db.py	DynamoDB put_item wrapper. Converts floats to Decimal (DynamoDB requirement). Module-level table handle, lazily initialised.
notifier.py	SNS publish. Per-type message templates (HR vs flame); severity label prefix; lazy SNS client.

6.4 Key Engineering Choices

Lazy global resource handles. Both `db._table` and `notifier._sns` are module-level globals initialised on first call. Lambda reuses container instances across invocations ("warm starts"); module-level state persists between calls, amortising the cost of SDK client creation. Cold starts pay the initialisation cost once.

`json.loads(json.dumps(alert), parse_float=Decimal)` in `db.store_alert`. DynamoDB's Python SDK rejects native Python float (it requires `decimal.Decimal` for exact arithmetic). This round-trip is the idiomatic conversion when the payload arrives as a Python dict with floats. Idempotency by composite key. The DynamoDB table uses (`device_id`, `timestamp`) as (HASH, RANGE). MQTT QoS 1 may deliver duplicate messages; `put_item` with the same key simply overwrites, yielding natural idempotency without explicit deduplication logic.

TTL for automatic GC. Each item has a `ttl` attribute set to `timestamp + 90` days. DynamoDB's TTL feature deletes expired items asynchronously, eliminating manual housekeeping.

Least-privilege IAM via SAM policy templates. Rather than hand-crafting IAM policies, the template uses SAM's `DynamoDBCrudPolicy` and `SNSPublishMessagePolicy`, scoped to the specific table and topic. This satisfies AWS's principle of least privilege without YAML verbosity.

IoT-to-Lambda permission boundary. A separate `AWS::Lambda::Permission` resource grants `iot.amazonaws.com` permission to invoke the Lambda, restricted by `SourceArn` to the specific Rules-Engine rule. This prevents arbitrary AWS services from triggering the function.

Severity-prefixed email subjects. Email subject lines are formatted as `Care360 Alert — CRITICAL/MEDIUM/LOW`, allowing recipients to triage at a glance or set up mail-client filters.

7. Tier 4: Dashboard

The dashboard/ directory contains a FastAPI backend (backend/main.py) and a Vite/React frontend. Three route modules are mounted under /api: alerts, config, status. The built React SPA is served from /.

8. End-to-End Alert Scenarios

8.1 Heart-rate Anomaly

T+0s Watch heart-rate sensor reports 135 BPM, accuracy 3
T+0s Reading buffered in HealthSensorManager
T+5s Batch loop: drainReadings() returns 1 HR + ~10 ACC readings
T+5s JSON serialised (~400 bytes), notify via BLE
T+5s Pi: on_batch called
T+5s unpack → 1 HR reading (135 BPM)
T+5s EnvCollector.read() → ENV appended
T+5s AlertDetector: streak = 1 (no alert)
T+10s Next batch, 138 BPM → streak = 2 (no alert)
T+15s Next batch, 132 BPM → streak = 3 → ALERT
T+15s LocalNotifier: 880 Hz beep + warning log
T+15s IoTPublisher.publish_alert via MQTT QoS 1
T+15s+ AWS IoT Rule matches → invokes Lambda
T+15s+ Lambda: validate → score (severity 2) → DynamoDB → SNS
T+~30s Caregiver receives email: "Care360 Alert — MEDIUM"

8.2 Flame Detection

Flame detection bypasses the 3-strike debounce — a single positive reading triggers an immediate FLAME_DETECTED alert at severity 3 (CRITICAL). The _flame_active flag prevents re-alerting while the flame persists; the flag resets when flame is no longer detected, allowing future episodes to alert again.

8.3 Watch Disconnection

When the Pi loses BLE contact with the watch:

async with BleakClient detects disconnect, raises BleakError.
_connect_and_listen propagates the exception.
run_forever's try/except catches it, logs a warning, sleeps 5 s.
Scan resumes; if the watch is back in range, connection re-establishes within ~30 s.

During the disconnect, the watch continues to buffer readings; once reconnected, the next batch carries the accumulated readings (potentially exceeding 490 bytes, triggering splitAndNotify recursion).

9. Known Limitations and Roadmap

Limitations

Single-device topology. Each Pi connects to one watch; the codebase assumes one-to-one. Multi-resident households would require concurrent BLE Central connections (technically feasible with bleak, not currently implemented).

No fall detection. Accelerometer data is collected and logged but not analysed for falls. Fall detection requires ML models not yet integrated.

No watch-side battery monitoring alert. Battery level is in the schema but unused on the Pi.

Hard-coded mock-hardware mode. Switching to real Sense HAT requires editing source rather than configuration.

Heart-rate test threshold (HR_HIGH_BPM = 75) shipped in config.py.

Production value of 120 must be restored before clinical use.

Roadmap

Stage	Item
Imminent	Restore production HR threshold;
Short-term	Hardware-mode toggle via environment variable; multi-watch support; SMS notification via SNS in addition to email.
Medium-term	Fall-detection ML model on accelerometer; battery alerts; encrypted BLE pairing.
Long-term	Federated learning on per-resident baseline heart-rate; remote configuration push from cloud to Pi; integration with hospital EHR systems.

10. Conclusion

Care360 demonstrates a pragmatic, defensible architecture for IoT health monitoring in environments where privacy and operational simplicity outweigh feature breadth. By keeping raw data on-device, performing detection at the edge, and using the cloud strictly as an alerting and audit substrate, the system achieves a credible balance between caregiver visibility and resident privacy.

The codebase shows several mature engineering patterns: contract-driven development (DATA_CONTRACT), symmetric tier architectures, three-layer error containment in the BLE client, stateful debounce for anomaly detection, and graceful degradation at every boundary. It also carries the typical traces

of an active, mid-stage project: a checked-in private key, a test-only heart-rate threshold, hard-coded device identifiers, and an incomplete dashboard. These are tracked items rather than design flaws.

Total scope across four tiers is approximately 1,100 lines of business code plus 200 lines of infrastructure YAML — a footprint small enough for a single engineer to hold in working memory, yet substantial enough to exercise the principal patterns of modern IoT engineering: BLE peripheral/central roles, foreground services, mTLS authentication, MQTT topic routing, serverless event handlers, and infrastructure-as-code.

Appendix A: Technology Stack Summary

Tier	Language	Runtime	Major Libraries
Watch	Kotlin	Wear OS / Android 14+	androidx.wear.compose, BluetoothGattServer
Gateway	Python 3.12+	Raspberry Pi (Linux ARM64)	bleak, paho-mqtt, sense-emu, gpiozero
Cloud	Python 3.12	AWS Lambda	boto3
Infrastructure	YAML	AWS SAM / CloudFormation	—
Dashboard	Python + TypeScript	FastAPI + React (Vite)	uvicorn, react-router